

Modul mahasiswa Lab 05 — Dockerfile dan Cloud Notes API

Kebijakan kelas: Lab ini latihan formatif, tanpa tugas, nilai, atau penyerahan terpisah. Satu proyek besar dikerjakan oleh kelompok **3 orang**, dengan presentasi checkpoint minggu 7 (UTS) dan hasil akhir minggu 14 (UAS). Simpan hasil lab hanya bila berguna sebagai referensi atau bukti proses proyek. Baca [brief proyek kelompok](#). Bobot resmi tetap mengikuti RPS/LMS.

Sesi RPS: 05 · **Jalur utama:** build dan run lokal · **Lanjutkan ke:** repo Lab 06 setelah praktik ini

Hasil belajar

Anda dapat menjelaskan tiap tahap `Dockerfile`, membangun image bertag, menguji REST API dalam container, menunjukkan proses berjalan sebagai user non-root, dan menjelaskan mengapa catatan hilang setelah container diganti. Kode FastAPI ada di `app/main.py`; dependensi ada di `requirements.txt`.

Teori singkat. Dockerfile adalah resep build image. Tahap `builder` memasang dependensi Python ke virtual environment; tahap `runtime` menyalin hanya hasil yang diperlukan serta kode aplikasi. `.dockerignore` mengurangi berkas yang masuk ke build context. `RUN` dieksekusi saat build; `CMD` memberi perintah default saat container mulai, sedangkan `ENTRYPOINT` dapat menetapkan executable utama. `ENV` pada Dockerfile berbeda dari `-e` pada `docker run`: `-e` memasok nilai saat container dijalankan. Data API ini disimpan di memori proses, belum di volume/database.

Persiapan dan build

Pastikan Docker Engine berjalan, port host **8000** kosong, dan terminal berada di root repo Lab 05. Di PowerShell, Bash, atau WSL:

```
Lab 05 · baca Dockerfile sebelum build

$ docker version --format '{{.Server.Version}}'
29.6.2

$ Select-String -Path Dockerfile -Pattern '^((FROM|RUN|COPY|USER|EXPOSE|CMD))'
Dockerfile:1:FROM python:3.12-slim AS builder
Dockerfile:3:COPY requirements.txt .
Dockerfile:4:RUN python -m venv /opt/venv && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt
Dockerfile:6:FROM python:3.12-slim AS runtime
Dockerfile:10:RUN useradd --create-home --uid 10001 appuser
Dockerfile:12:COPY --from=builder /opt/venv /opt/venv
Dockerfile:13:COPY app ./app
Dockerfile:14:USER appuser
Dockerfile:15:EXPOSE 8000
Dockerfile:16:CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

Output aktual: FROM membentuk tahap image; RUN saat build; USER/CMD dipakai saat container dimulai.
```

Perintah: `docker version --format '{{.Server.Version}}'` dan PowerShell `Select-String -Path Dockerfile -Pattern '^((FROM|RUN|COPY|USER|EXPOSE|CMD))'`. **Fungsi:** memastikan Engine aktif dan membaca tahap build/runtime sebelum build. **Cara kerja:** pencarian memilih instruksi Dockerfile; `FROM` membuat tahap, `RUN` memasang dependensi, `USER` memilih proses non-root, `CMD` menjalankan API. **Baca hasil:** dua `FROM`, `USER appuser`, dan port `8000`. Ini render output command aktual.

```
docker version
docker build -t cloud-notes-api:lab05 .
docker image ls cloud-notes-api
docker run --name cloudlab-api -d -p 127.0.0.1:8000:8000 -e APP_ENV=classroom cloud-notes-api:lab05
docker ps --filter name=cloudlab-api
docker logs cloudlab-api
docker exec cloudlab-api id
docker image inspect cloud-notes-api:lab05 --format '{{.Size}} bytes; user={{.Config.User}}'
```

Checkpoint A: image `cloud-notes-api:lab05` terdaftar; log menunjukkan Uvicorn menerima koneksi pada port 8000. Tunggu beberapa detik bila API belum siap segera setelah `docker run`. Buka `http://127.0.0.1:8000/docs`. Dokumentasi OpenAPI yang Anda lihat akan mirip screenshot hasil run lokal berikut.

Cloud Services · hasil perintah lokal

Image build dan user runtime

Output aktual dari sesi uji, ditata ulang agar mudah dibaca pada modul.

```
docker image inspect; docker exec id; docker ps
```

```
cloud-notes-api:lab05 | user=appuser | 52164335 bytes
```

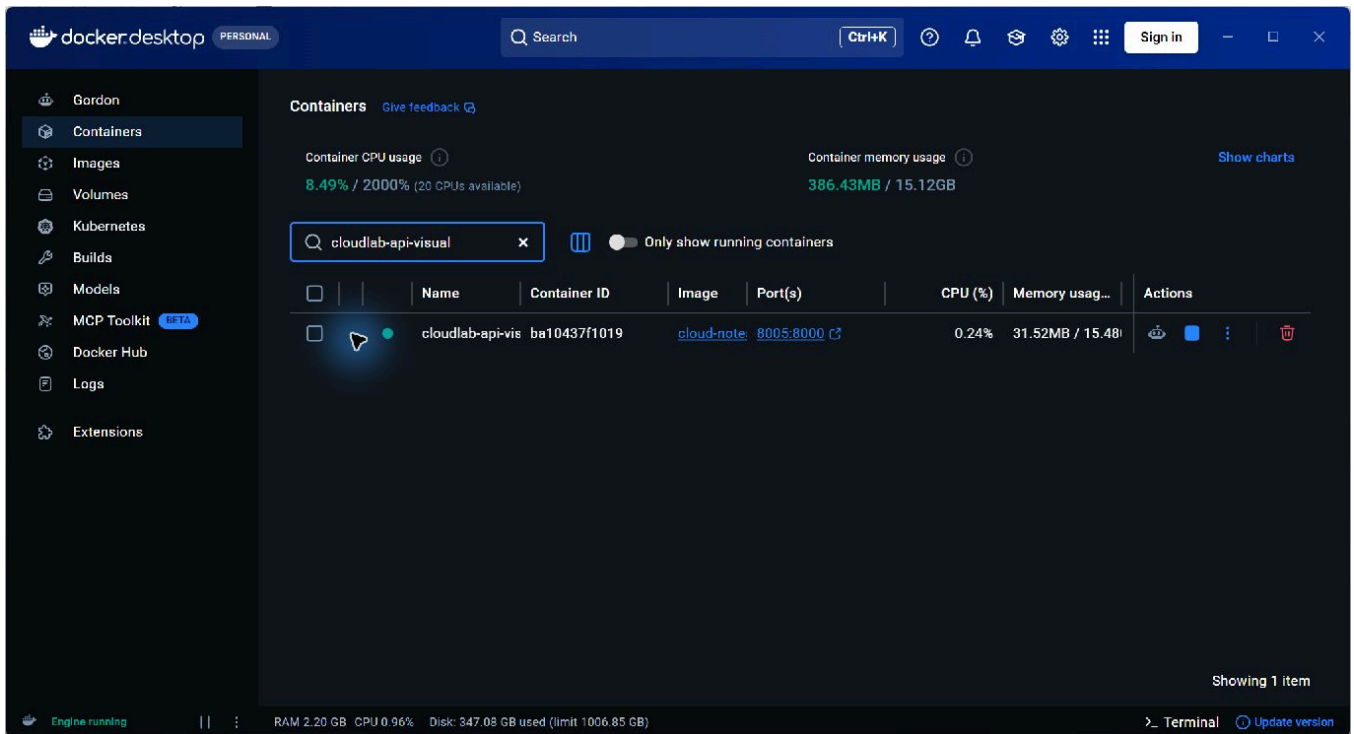
```
docker exec cloudlab-api id
```

```
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
```

NAMES	IMAGE	PORTS
cloudlab-api	cloud-notes-api:lab05	127.0.0.1:8000->8000/tcp

- **Langkah:** Jalankan `docker build -t cloud-notes-api:lab05 .`, `docker exec cloudlab-api id`, dan `docker image inspect`. **Fungsi:** Membuktikan build API, user runtime non-root, serta port yang dipakai. **Cara kerja:** Dockerfile multi-stage membuat image; proses API berjalan sebagai appuser di dalam container. **Baca hasil:** Baca tag image, UID 10001/appuser, dan mapping host 8000 ke container 8000; ukuran image dapat berbeda.

Gambar: hasil aktual `docker image inspect`, `docker exec id`, dan `docker ps`. Pada mesin Anda, periksa appuser, UID 10001, dan port host 8000; ukuran image dapat berbeda.



- **Langkah:** Buka Docker Desktop > Containers setelah `docker run`, lalu bandingkan dengan `docker ps`.
Fungsi: Melihat status container dan pemetaan port tanpa kehilangan arti perintah CLI. **Cara kerja:** Docker menjalankan image `cloud-notes-api:lab05` sebagai container; titik hijau menandai proses sedang berjalan.
Baca hasil: Pada screenshot uji, host **8005** diteruskan ke port container **8000** (`docker run --name cloudlab-api-visual -d -p 127.0.0.1:8005:8000 ...`) karena Lab 06 memakai host 8000 pada waktu yang sama. Langkah utama Anda tetap memakai `127.0.0.1:8000:8000` dan URL port 8000; status HTTP 200 diuji terpisah agar titik hijau tidak disalahartikan sebagai healthcheck.*

Cloud Notes 0.5 OAS 3.1

/openapi.json

default

GET	/health	Health	▼
GET	/runtime	Runtime	▼
GET	/notes	List Notes	▼
POST	/notes	Create Note	▼
GET	/notes/{note_id}	Get Note	▼

Schemas

HTTPValidationError > Expand all object

NoteInput > Expand all object

ValidationError > Expand all object

- **Langkah:** Setelah `docker run -p 127.0.0.1:8000:8000`, buka `http://127.0.0.1:8000/docs`. **Fungsi:** Mengenali kontrak endpoint API Cloud Notes. **Cara kerja:** FastAPI membangun Swagger UI dari skema OpenAPI aplikasi yang hidup di container. **Baca hasil:** Cari route `/health`, `/runtime`, dan `/notes`; buka satu route untuk melihat metode serta bentuk respons.

Praktik API dan validasi

Di PowerShell:

```
Invoke-RestMethod http://127.0.0.1:8000/health
Invoke-RestMethod http://127.0.0.1:8000/runtime
Invoke-RestMethod http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body
'{"title":"Docker","content":"Catatan kelas"}'
Invoke-RestMethod http://127.0.0.1:8000/notes
try { Invoke-WebRequest http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body
'{"title":"","content":"uji"}' -UseBasicParsing } catch { [int]$_ .Exception.Response.StatusCode }
```

Di Bash/WSL:

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/runtime
curl -fsS -X POST http://127.0.0.1:8000/notes -H 'Content-Type: application/json' -d
'{"title":"Docker","content":"Catatan kelas"}'
curl -fsS http://127.0.0.1:8000/notes
curl -s -o /dev/null -w '%{http_code}\n' -X POST http://127.0.0.1:8000/notes -H 'Content-Type:
application/json' -d '{"title":"","content":"uji"}'
```

Checkpoint B: `/health` bernilai `ok`, `/runtime` menampilkan `APP_ENV=classroom` dan `storage=memory`, POST membuat catatan, GET menampilkannya, dan judul kosong menghasilkan HTTP **422**. ID dan urutan catatan bisa berbeda. Screenshot berikut adalah contoh respons dari container lokal; buat bukti sendiri saat praktik.



The screenshot shows a terminal window titled "Lab 05: respons API lokal". It displays the results of several REST client commands. The first command is `Invoke-RestMethod /health, /runtime, POST /notes, GET /notes`. Below this, the results of individual commands are shown: `GET /health` returns `{"status":"ok","service":"cloud-notes-api"}`; `GET /runtime` returns `{"environment":"classroom","storage":"memory"}`; `POST /notes` returns `{"id":1,"title":"Docker","content":"Catatan kelas"}`; and `GET /notes` returns `{"value":[{"id":1,"title":"Docker","content":"Catatan kelas"}],"Count":1}`.

- **Langkah:** Kirim GET `/health`, GET `/runtime`, POST `/notes`, dan GET `/notes` dengan `curl` atau `Invoke-RestMethod`. **Fungsi:** Menguji kesehatan, identitas runtime, dan alur tulis/baca catatan. **Cara kerja:** Request

host masuk melalui port publish ke FastAPI; POST menyimpan catatan dalam memori proses. **Baca hasil:** Baca `status` sehat, `appuser`, respons catatan baru, lalu item yang sama pada daftar GET.

Hapus container, jalankan image yang sama lagi, lalu GET daftar catatan. Gunakan perintah sesuai terminal:

```
docker rm -f cloudlab-api
docker run --name cloudlab-api -d -p 127.0.0.1:8000:8000 -e APP_ENV=classroom cloud-notes-api:lab05
```

PowerShell: `Invoke-RestMethod http://127.0.0.1:8000/notes`; Bash: `curl -fsS http://127.0.0.1:8000/notes`.

Checkpoint C: daftar dimulai kosong setelah proses/container baru siap. Jelaskan penyebabnya dan bandingkan dengan Lab 06 nanti.

Cloud Services · hasil perintah lokal

Validasi dan reset data memori

Output aktual dari sesi uji; ditata ulang agar mudah dibaca pada modul.

POST /notes; input invalid; docker rm/run; GET /notes

POST /notes sebelum container diganti
{ "id":1, "title":"Sementara", "content":"Hilang saat container diganti" }

POST /notes dengan title kosong
HTTP 422

GET /notes setelah docker rm + docker run
[]

- **Langkah:** POST judul kosong ke `/notes`, lalu `docker rm -f cloudlab-api` dan jalankan container baru dari image sama. **Fungsi:** Mendemonstrasikan validasi input dan batas penyimpanan in-memory. **Cara kerja:** FastAPI menolak payload tidak valid; container baru memulai proses dengan memori kosong. **Baca hasil:** Cari HTTP 422 dan daftar catatan kosong setelah restart lewat container baru.

Gambar: output aktual dari POST valid, POST berjudul kosong, dan GET setelah `docker rm/docker run`. ID serta isi catatan Anda boleh berbeda.

Tantangan kode mandiri: tambahkan endpoint `GET /stats` pada salinan Anda di `app/main.py`. Kembalikan `note_count` dan `longest_title_chars`; saat belum ada catatan keduanya bernilai 0. Gunakan lock yang sama dengan operasi catatan, uji sebelum dan sesudah membuat dua catatan, lalu build ulang image dengan tag berbeda. Tantangan ini latihan tambahan; endpoint inti di atas harus berhasil lebih dahulu.

Analisis, laporan, dan Git

Jawab tanpa menyalin contoh: (1) bagian mana dari Dockerfile berjalan saat build dan bagian mana saat run; apa beda `CMD` dan `ENTRYPOINT` dalam konteks ini; (2) mengapa `.dockerignore` memuat `.env`; (3) apa arti port kiri/kanan pada `127.0.0.1:8000:8000`; (4) apa hubungan validasi `Field(min_length=1)` dengan HTTP 422; (5) apa kelemahan menyimpan catatan hanya di memori.

Dari root repo, salin [template laporan](#) menjadi `hasil/lab05.md`: PowerShell `Copy-Item hasil/TEMPLATE_LAPORAN.md hasil/lab05.md`; Bash `cp hasil/TEMPLATE_LAPORAN.md hasil/lab05.md`. Isi hasil nyata, perintah, jawaban, dan

screenshot Anda di `hasil/bukti/`. Jangan memakai screenshot modul sebagai bukti pribadi.

```
docker rm -f cloudlab-api
```

Dari root repo Lab 05, ikuti [panduan Git](#) untuk repo sendiri:

```
git status --short
git add Dockerfile .dockerignore requirements.txt app MODUL_MAHASISWA.md hasil/lab05.md hasil/bukti
git diff --cached --name-only
git diff --cached --check
git commit -m "lab05: build dan uji Cloud Notes API"
git push
```

Lab 05 · cek repo Git terbit

```
$ git remote -v
origin https://github.com/SeedFlora/meet5CloudService.git (fetch)
origin https://github.com/SeedFlora/meet5CloudService.git (push)
```

```
$ git status --short
?? screenshots/lab05_dockerfile_persiapan.png
```

```
$ git log -1 --oneline
ace4300 Publish complete Lab 05 Dockerfile modules
```

```
$ git rev-parse HEAD; git ls-remote origin refs/heads/main
local: ace4300f75c623dbb3aa8bb2528350484170f48f
remote: ace4300f75c623dbb3aa8bb2528350484170f48f
Sama: True
```

Output aktual repo template SeedFlora. Mahasiswa melakukan commit/push pada repo pribadi.

SHA pada gambar adalah snapshot saat uji. Setelah modul diperbarui, jalankan ulang perintah untuk memeriksa commit terbaru.

Perintah: `git remote -v`, `git status --short`, `git log -1 --oneline`, `git rev-parse HEAD`, dan `git ls-remote origin refs/heads/main`. Fungsi: memeriksa tujuan serta hasil push kode. Cara kerja: Git membandingkan SHA commit lokal dengan SHA `main` remote. Baca hasil: `Sama: True` untuk repo pengajar; mahasiswa mengulangnya pada repo pribadi. Ini render output command aktual.

Jika belum ada folder bukti, hilangkan argumen itu dari `git add`. Pastikan staged tidak berisi `.env`, kredensial, atau hasil build. Jika push pertama belum punya upstream, jalankan `git push -u origin main`.

Jika gagal: konflik port 8000 biasanya berarti container Lab 06 atau Lab 05 lama masih aktif (`docker ps`). Jika `docker build` gagal mengambil image/dependensi, cek koneksi internet dan jalankan ulang. Jika aplikasi belum siap setelah `run`, tunggu status log Uvicorn sebelum memanggil API.

Pilihan cloud — Docker Hub: bila dosen meminta publish image dan Anda punya akun, jalankan `docker tag cloud-notes-api:lab05 NAMA_AKUN/cloud-notes-api:lab05`, `docker login`, lalu `docker push NAMA_AKUN/cloud-notes-api:lab05`. Ganti `NAMA_AKUN`; jangan merekam token login atau memasukkannya ke repo. Push Git untuk kode tetap terpisah dari push image ke registry.

Kunci tantangan dan kasus kerja harian

Kasus: tim support perlu melihat jumlah tiket/catatan serta judul terpanjang tanpa membuka semua isi catatan. Anda menambahkan endpoint statistik baca saja. Kunci ada di bawah; lakukan percobaan sendiri dahulu, lalu bandingkan hasil. Semua perintah dijalankan dari **root repo Lab 05**.

1. Baca kontrak aplikasi

Dockerfile tahap `builder` memasang paket ke `/opt/venv`; tahap `runtime` menyalin hasilnya, memasang kode, dan menjalankan Uvicorn sebagai UID 10001. `docker build` menjalankan instruksi build dan menghasilkan image. `docker run` membuat proses container dari image tersebut. Port kiri `127.0.0.1:8000` adalah port laptop yang hanya menerima koneksi lokal; port kanan `8000` milik proses dalam container.

2. Kunci kode `GET /stats`

Tambahkan potongan berikut ke akhir `app/main.py`, di luar fungsi lain. Variabel `_notes` dan `_lock` sudah disediakan oleh starter; jangan membuat dictionary atau lock baru karena hasilnya akan terpisah dari endpoint `/notes`.

```
@app.get("/stats")
def stats() -> dict[str, int]:
    with _lock:
        titles = [item["title"] for item in _notes.values()]
        return {
            "note_count": len(titles),
            "longest_title_chars": max((len(title) for title in titles), default=0),
        }
```

Lock memastikan snapshot daftar judul konsisten bila ada request POST bersamaan. `default=0` membuat hasil awal tetap valid ketika belum ada catatan. Hitung panjang judul yang sudah divalidasi dan dipangkas spasinya oleh `NoteInput`.

3. Build ulang dan uji dari awal sampai akhir

Gunakan tag baru supaya jelas image mana yang berisi solusi. Hapus container percobaan Lab 05 yang lama bila masih ada, lalu:

```
docker build -t cloud-notes-api:stats .
docker rm -f clouddlab-api
docker run --name clouddlab-api -d -p 127.0.0.1:8000:8000 -e APP_ENV=classroom cloud-notes-api:stats
```

Jika `docker rm` melaporkan container tidak ada, lanjutkan ke `docker run`. Tunggu `/health` menjawab HTTP 200. Di **PowerShell**, jalankan:

```
Invoke-RestMethod http://127.0.0.1:8000/stats
Invoke-RestMethod http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body
'{"title":"Ops","content":"Pertama"}'
Invoke-RestMethod http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body
'{"title":"Incident 42","content":"Kedua"}'
Invoke-RestMethod http://127.0.0.1:8000/stats
try { Invoke-WebRequest http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body
'{"title":"  ","content":"uji"}' -UseBasicParsing } catch { [int]$_ .Exception.Response.StatusCode }
```

Di **Bash/Git Bash**, padanannya:

```
curl -fsS http://127.0.0.1:8000/stats
curl -fsS -X POST http://127.0.0.1:8000/notes -H 'Content-Type: application/json' -d
'{"title":"Ops","content":"Pertama"}'
curl -fsS -X POST http://127.0.0.1:8000/notes -H 'Content-Type: application/json' -d
'{"title":"Incident 42","content":"Kedua"}'
curl -fsS http://127.0.0.1:8000/stats
curl -s -o /dev/null -w '%{http_code}\n' -X POST http://127.0.0.1:8000/notes -H 'Content-Type:
application/json' -d '{"title":"  ","content":"uji"}'
```

Hasil kunci: awal `{"note_count":0,"longest_title_chars":0}`; sesudah dua POST `{"note_count":2,"longest_title_chars":11}`; judul spasi saja ditolak HTTP **422**. Jika hasil `/stats` masih 404, image baru belum dibuild atau container masih memakai tag lama. Cek dengan `docker ps --filter name=cloudlab-api --format '{{.Image}}'`. Setelah merekam bukti, jalankan `docker rm -f cloudlab-api`.

```
Lab 05 - Docker API aktual
docker build cloud-notes-api:lab05 -> sukses
docker run -p 127.0.0.1:8000:8000 -> cloudlab-api aktif
GET /health -> HTTP 200 {'status': 'ok', 'service': 'cloud-notes-api'}
GET /runtime -> HTTP 200 {'environment': 'classroom', 'storage': 'memory'}
POST /notes -> HTTP 201 id=1 title=Docker
GET /notes -> HTTP 200 count=1
POST /notes title="  " -> HTTP 422
GET /notes/999999 -> HTTP 404
```

Output HTTP aktual ditata ulang agar terbaca. Password/token tidak direkam.

Perintah: `docker build`, `docker run`, `GET health/runtime`, `POST` dan `GET notes`, lalu uji judul spasi. Cuplikan hasil Docker dan HTTP aktual ditata agar terbaca. HTTP 201 berarti catatan dibuat; HTTP 422 menunjukkan validasi; container baru mengembalikan daftar kosong karena data hanya di memori.

Cloud Notes 0.5 OAS 3.1

/openapi.json

default

GET	/health	Health	▼
GET	/runtime	Runtime	▼
GET	/notes	List Notes	▼
POST	/notes	Create Note	▼
GET	/notes/{note_id}	Get Note	▼

Schemas

HTTPValidationError > Expand all object

NoteInput > Expand all object

ValidationError > Expand all object

Perintah: buka `http://127.0.0.1:8000/docs` setelah `docker run`. Browser mengambil Swagger UI dari API yang sedang hidup. Gunakan route `/health`, `/runtime`, dan `/notes` untuk membaca kontrak sebelum memanggil endpoint.

```
Lab 05 - solusi /stats, image dibuild dari salinan kunci
GET /stats sebelum POST -> HTTP 200, 0 catatan, 0 karakter
POST /notes (Ops) -> HTTP 201
POST /notes (Incident 42) -> HTTP 201
GET /stats -> HTTP 200, 2 catatan, judul terpanjang 11
POST judul berisi spasi saja -> HTTP 422
```

Hasil Docker/API aktual ditata ulang agar terbaca; kode starter tetap tanpa endpoint tantangan.

Perintah: tambah kode `/stats` di salinan uji, build image `stats-proof`, lalu GET sebelum/sesudah dua POST. Gambar adalah hasil API aktual yang ditata agar terbaca: jumlah 0→2, judul terpanjang 11 karakter, dan input kosong HTTP 422. Starter repo tetap menyisakan tantangan untuk dikerjakan sendiri.

4. Kunci lima pertanyaan analisis

1. `RUN` dan `COPY` mengubah image saat build; `CMD` dijalankan saat container start. `ENTRYPOINT` dapat menetapkan executable utama yang menerima argumen dari `CMD`; lab ini cukup dengan `CMD` `Uvicorn`.
2. `.dockerignore` mencegah `.env` masuk build context sehingga konfigurasi lokal tidak ikut image. File rahasia tetap harus diabaikan Git.
3. Pada `127.0.0.1:8000:8000`, bagian kiri adalah alamat dan port laptop; bagian kanan adalah port aplikasi dalam container. Loopback membatasi akses dari host lain.
4. `Field(min_length=1)` menolak string kosong. Validator tambahan `reject_blank_text` memangkas spasi dan menolak teks yang hanya berisi spasi; FastAPI memberi 422 karena payload gagal validasi.
5. `_notes` hidup di memori proses. Mengganti container membuat memori baru dan daftar kosong. Lab 06 memindahkan sumber data ke PostgreSQL dalam named volume.